COMPILER DESIGN

ASSIGNMENT

Name : SUDHARSAN D

Register Number : 192524063

Course Code : CSA1405

Course Name : Compiler Design

Slot : C

Institution : SIMATS ENGINEERING

Faculty Name : Dr. S. Sivagami

25/07/26 CO2 Design task

ASSESSMENT TOOL 4

1. Design an unambiguous context-free grammar (CFG) to generate arithmetic expressions consisting of the symbols id, *, -, (), ↑ (where ↑ represents exponentiation), ensuring correct operation precedence and associativity.

   Factor, design a shift-reduce parser for the given grammar and demonstrate its capability by parsing the input string :

   id - id * id ↑ id (id ↑ id) ↑ id

   (i) Clearly specify production rules that eliminate ambiguity.

   (ii) Handle operator precedence (↑ > * > -) and associativity.

   (iii) Show step-by-step shift-reduce parsing actions.

   (iv) Indicate reductions and handle parsing decision properly.

Answers

(i) Unambiguous Grammar

E → Expression

T → Term

P → Power

F → Factor

Production Rules :

1. $E \rightarrow E - T$
2. $E \rightarrow T$
3. $T \rightarrow T * P$
4. $T \rightarrow P$
5. $P \rightarrow F \uparrow P$
6. $P \rightarrow F$
7. $F \rightarrow id$
8. $F \rightarrow (E)$

The grammar is unambiguous because

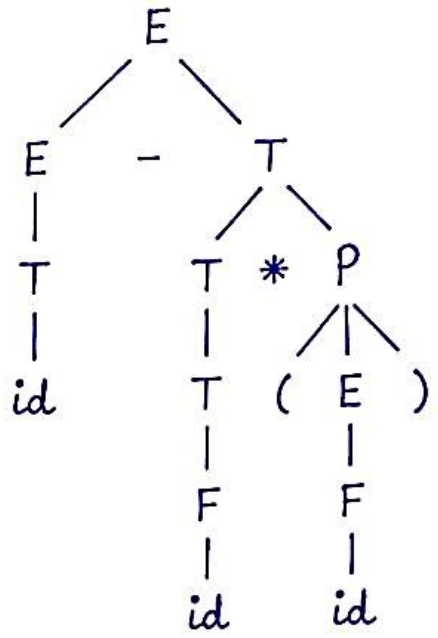
* Exponentiation has highest precedence
* Multiplication has second precedence
* Substraction has lowest precedence
* $\uparrow$ is right associative
* and $-$ are left associative

Input string :

$id - id * id \uparrow id$

Stack	i / P buffer	Action shift	shift / Reduce
$	id - id * id ↑ id $	Reduce id → F	~~[illegible]~~
$ id	- id * id ↑ id $	Reduce F → P	E → E - T
$ F	- id * id ↑ id $	Reduce P → T	E → T
$ P	- id * id ↑ id $	Reduce T → E	T → T * P
$ T	- id * id ↑ id $	Shift :	T → P
$ E	- id * id ↑ id $	Shift -	P → F | P
$ E -	id * id ↑ id $	Reduce id → F	P → F
$ E - id	id * id ↑ id $	Reduce F → P, P → T	F → id
$ E - F	* id ↑ id $	Reduce E - T → E	F → (E)
$ E - T	* id ↑ id $	Reduce E → T, T → P, P → F.	
$ E	* id ↑ id $	~~Shift reduce~~.	id -
$ F	* id ↑ id $	Reduce by	-
~~$ E $~~	* id ↑ id $	Reduce	-
$	↑ id $	shift	
$ E → T * F	↑ id $	shift	
$ E → T * F ↑	↑ id $		
$ E - T * F ↑ id	id $		
	id $		
	$		

Parse Tree



Result

id - id * id ↑ id

2. Design an LL(1) Parser for the given grammar

D → TL ;

L → L , id | id

T → int | float

(i) Transform the grammar by eliminating left recursion (if present) and ensure it is suitable for LL(1) parsing.

(ii) Design and compute FIRST and FOLLOW sets for all non-terminals in the transformed grammar.

(iii) Construct the LL(1) parsing table based on the computed FIRST and FOLLOW sets.

(iv) Demonstrate the working of the parser by parsing 'true' for the given input.

Answer

$D \rightarrow TL\ ;$

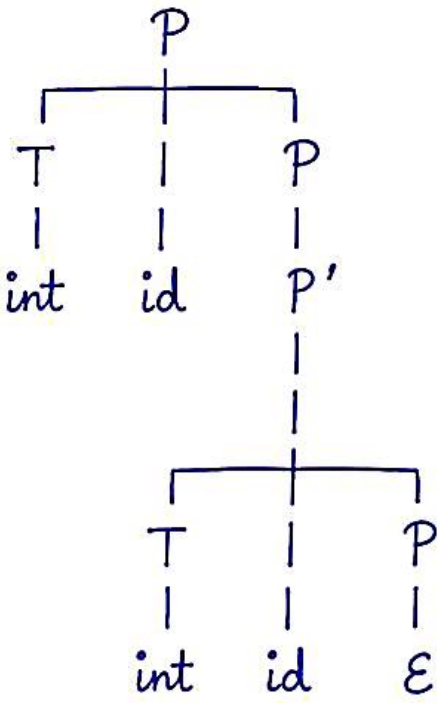
$L \rightarrow L\ ;\ id \mid id$

$T \rightarrow int \mid float$

Parsing process :

Stack	Input	Action
$\$D$	int id , id ; $	$D \rightarrow TL$
$\$;LT$	int id , id ; $	Expand D
$\$;Lint$	int id , id ; $	$T \rightarrow int$
$\$;Lid$	int id , id ; $	match int
$\$;L$	id , id ; $	$L \rightarrow L'id'$
$\$;L'id$	id , id ; $	Expand
$\$;L'id$	id , id ; $	match id
$\$;L'$	, id ; $	$L' \rightarrow ,idL'$
$\$;L'id$	, id ; $	Expand
$\$;L'id$	, id ; $	match ,
$\$;L'id$	id ; $	match id
$\$;L'$	id ; $	$L' \rightarrow \epsilon$
$\$;$	; $	POP L'
$\$;$	; $	match ;
$\$;$	$	Accept

Parse Tree



3. Design and implement a recursive descent parser for the following grammar.

$S \rightarrow AB \mid BC$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

(i) Development of procedures for each non-terminal (S, A, B, C)

(ii) Application of top-down parsing strategy with backtracking (if possible)

(iii) Proper handling of input string matching using a recursive approach.

Solution

Grammar

$S \rightarrow AB|BC$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

FIRST Sets :

$FIRST(A) = \{a\}$, $FIRST(B) = \{b\}$; $FIRST(C) = \{c\}$

$FIRST(S) = \{a, b\}$.

Procedures :

S() : If lookahead = 'a' then A() ; B() ; else if lookahead = 'b' then BC() ; C() else error

A() : match ('a')

B() : match ('b')

C() : match ('c')

Trace for 'ab' :

1. $S \rightarrow AB$
2. match a
3. match b
4. Accept.

Trace for 'bc' :

1. $S \rightarrow BC$
2. match b
3. match c
4. Accept.

Accepted : ab, bc

Rejected : ac, ba, a a, bb, abc.